

Математическая модель и алгоритмы backend

cosmo_satellite

Разбор фактически реализованного кода: геометрия, графовая модель, маршрутизация, отказоустойчивость, динамика и сравнение вариантов

Состояние репозитория:	839003823a30
Дата разбора:	12 сентября 2026
Проверка:	148 backend-тестов проходят

Документ описывает вычислительную модель математически. JSON-кодеки, DTO, frontend-проекции и транспортные детали рассматриваются только как границы данных и не включаются в алгоритмическое ядро.

Contents

1	Общая композиция модели	3
1.1	Обозначения сложности	3
2	Пространственная модель	4
2.1	Константы reference-case	4
2.2	Доступность спутника и шлюза	4
2.3	Круговая орбита	4
2.4	Наземная точка на сферической Земле	5
2.5	Наземное наблюдение и угол места	5
2.6	Межспутниковый контакт и экранирование Землёй	5
2.7	Сложность одного пространственного снимка	6
3	Статическая графовая модель	6
3.1	Физический граф и quegu-граф	6
3.2	Покрытие и достижимость — разные величины	7
3.3	Допустимые первые спутники	7
3.4	Диагностика отсутствия маршрута	7
4	Маршрутизация	8
4.1	Детерминированный Дейкстра	8
4.2	Минимум переходов	8
4.3	Минимальная геометрическая длина	8
4.4	Структурированное качество и частичный порядок	9
4.5	Отказоустойчивый маршрут resilient_distance	9
5	Мгновенная отказоустойчивость	10
5.1	Спутниковая vertex-connectivity	10
5.2	Проверка N-1 перебором отказов	10
6	Мгновенный контрфактический анализ отказа спутника	11
6.1	Статический рейтинг критичности	11
6.2	Главная стоимость статического анализа	11
7	Динамическая модель	12
7.1	Дискретная временная сетка	12
7.2	Availability и outage-интервалы	12
7.3	Статистика причин отсутствия маршрута	12

7.4	Эпизоды маршрута и переключения	13
7.5	Временная N-1 устойчивость	13
8	Периодическая критичность спутников	13
8.1	Контрфактическая доступность	13
8.2	Outage-ущерб	14
8.3	Интегральные потери резервирования	14
8.4	Влияние на маршруты	14
8.5	Периодический рейтинг спутников	15
8.6	Сложность динамического анализа	15
9	Сравнение вариантов	15
9.1	Итоговые метрики варианта	16
9.2	Различие маршрутов между вариантами	16
10	Что является заменяемым математическим компонентом	16
11	Что текущий backend математически не моделирует	17
12	Сводная таблица сложности	18
13	Краткие выводы по математике реализации	18
A	Трассировка формул к backend-модулям	19

1 Общая композиция модели

Бэкенд реализует композицию четырёх математических уровней:

сценарий $\xrightarrow{3D}$ снимок контактов $\xrightarrow{\text{static}}$ графовый анализ,
 графовый анализ по $t_k \xrightarrow{\text{dynamic}}$ временные метрики $\xrightarrow{\text{comparison}}$ сравнение вариантов.

Ключевое свойство архитектуры: каждый уровень отвечает на отдельный вопрос.

1. **Spatial 3D:** где находятся узлы и существует ли между парой узлов допустимый *прямой* контакт?
2. **Static model:** существует ли в зафиксированном снимке путь от клиента до доступного шлюза, какой путь выбрать и насколько сеть устойчива к отказам?
3. **Dynamic model:** как эти свойства меняются на дискретной временной сетке, каковы availability/outage, переключения маршрутов и интегральная критичность спутников?
4. **Variant comparison:** чем две готовые динамические реализации отличаются по исходным параметрам и результатам?

Временная модель не интерполирует графовые состояния: на каждом t_k заново строится 3D-снимок, затем статический графовый анализ. Это важно: доступность, контакты и маршруты считаются дискретными фактами конкретного снимка.

1.1 Обозначения сложности

Далее используются обозначения:

N	число всех спутников в сценарии;
S	число активных спутников в конкретный момент времени ($S \leq N$);
G	число наземных узлов; $G = C + W$;
C	число клиентов; W — число шлюзов;
V	число узлов статического графа, порядка $N + G$;
E	число прямых сетевых контактов в статическом графе;
F	число доступных спутников в домене одиночных отказов; обычно $F \leq S$;
K	число включённых стратегий маршрутизации; в reference-плане $K = 2$;
T	число временных отсчётов;
d_c	число первых соседей клиента c в ориентированном query-графе;
$D(V, E)$	стоимость одного запуска Дейкстры; номинально $\mathcal{O}((V + E) \log V)$;
$M(V, E)$	стоимость min-cut после vertex splitting.

Для NetworkX 3.6.1 используемый по умолчанию highest-label preflow-push имеет заявленную сложность $\mathcal{O}(V^2\sqrt{E})$. После vertex splitting размер графа меняется только на постоянный множитель по V и на добавочные $\mathcal{O}(V + W)$ рёбра.

2 Пространственная модель

2.1 Константы reference-case

В адаптере сценария зафиксированы

$$R = 6371 \text{ км}, \quad \mu = 398600.435507 \text{ км}^3/\text{с}^2, \quad T_E = 86164.09054 \text{ с}.$$

Остальные параметры приходят из сценария: высота орбиты h , наклонение i , начальный угол вращения Земли θ_0 , минимальный угол места $\varepsilon_{\min}$ и максимальная дальность межспутниковой связи $D_{\max}$.

2.2 Доступность спутника и шлюза

Для спутника s с номером запуска b_s и текущей стадией развёртывания L :

$$A_s(t) = \mathbf{1}[b_s \leq L] \prod_{I \in \mathcal{F}_s} \mathbf{1}[t \notin I],$$

где каждый интервал отказа полуоткрыт: $I = [t_{\text{start}}, t_{\text{end}})$.

Для клиента доступность всегда равна 1. Для шлюза g :

$$A_g(t) = \prod_{I \in \mathcal{O}_g} \mathbf{1}[t \notin I].$$

Позиция спутника вычисляется независимо от $A_s(t)$: отказавшийся спутник остаётся геометрическим объектом, но исключается из генерации сетевых контактов.

2.3 Круговая орбита

Радиус орбиты:

$$r = R + h.$$

Среднее угловое движение:

$$n = \sqrt{\frac{\mu}{r^3}}.$$

Для спутника s в плоскости p с RAAN Ω_p , фазой φ_p и угловым слотом σ_s :

$$u_s(t) = \sigma_s + \varphi_p + nt.$$

Все углы в вычислениях переводятся в радианы.

Инерциальные координаты:

$$\begin{aligned} x_I &= r(\cos \Omega \cos u - \sin \Omega \sin u \cos i), \\ y_I &= r(\sin \Omega \cos u + \cos \Omega \sin u \cos i), \\ z_I &= r \sin u \sin i. \end{aligned}$$

Угол вращения Земли:

$$\theta(t) = \theta_0 + \frac{2\pi t}{T_E}.$$

Переход к Earth-fixed системе:

$$\begin{bmatrix} x_E \\ y_E \\ z_E \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_I \\ y_I \\ z_I \end{bmatrix}.$$

Для одного спутника вычисление имеет $\mathcal{O}(1)$ времени и памяти; для всех N — $\mathcal{O}(N)$ без учёта проверки списков отказов.

2.4 Наземная точка на сферической Земле

Для широты ϕ и долготы λ :

$$\mathbf{g} = R \begin{bmatrix} \cos \phi \cos \lambda \\ \cos \phi \sin \lambda \\ \sin \phi \end{bmatrix}.$$

2.5 Наземное наблюдение и угол места

Пусть $\mathbf{s}$ — Earth-fixed позиция спутника, $\mathbf{g}$ — позиция наземной точки,

$$\Delta = \mathbf{s} - \mathbf{g}, \quad d = \|\Delta\|.$$

Тогда в реализации

$$\sin \varepsilon = \frac{\Delta \cdot \mathbf{g}}{dR}, \quad \varepsilon = \arcsin \left(\text{clip} \left(\frac{\Delta \cdot \mathbf{g}}{dR}, -1, 1 \right) \right).$$

При $d = 0$ используется $\varepsilon = 90^\circ$ как защитный частный случай.

Геометрическая видимость:

$$V_{g,s}(t) = \mathbf{1}[\varepsilon_{g,s}(t) \geq \varepsilon_{\min}].$$

Прямой наземно-спутниковый сетевой контакт в reference-политике существует тогда и только тогда, когда спутник активен, наземный узел доступен и пара геометрически видима:

$$L_{g,s}(t) = A_s(t)A_g(t)V_{g,s}(t).$$

Важно: для клиента $A_g(t) = 1$; outage наземного узла в текущей модели относится к шлюзам.

2.6 Межспутниковый контакт и экранирование Землёй

Для активных спутников i, j :

$$\Delta = \mathbf{r}_j - \mathbf{r}_i, \quad d_{ij} = \|\Delta\|.$$

Ближайшая к центру Земли точка отрезка $[\mathbf{r}_i, \mathbf{r}_j]$ находится через

$$q^* = \text{clip} \left(-\frac{\mathbf{r}_i \cdot \Delta}{\|\Delta\|^2}, 0, 1 \right), \quad \rho_{ij} = \|\mathbf{r}_i + q^* \Delta\|.$$

Если позиции совпали, берётся $\rho_{ij} = \|\mathbf{r}_i\|$.

Reference-условие ISL строгое по обеим границам:

$$L_{ij}(t) = \mathbf{1}[d_{ij} < D_{\max}] \mathbf{1}[\rho_{ij} > R].$$

То есть касание сферы $\rho = R$ уже считается недопустимым, как и расстояние равно $D_{\max}$.

2.7 Сложность одного пространственного снимка

Текущий broad phase перебирает *все* пары активных спутников:

$$\binom{S}{2} = \Theta(S^2).$$

Для каждого наземного узла вычисляется наблюдение каждого активного спутника: $\Theta(GS)$.

Без учёта поиска outage-интервалов:

$$T_{3D} = \Theta(N + S^2 + GS), \quad M_{3D} = \mathcal{O}(N + G + S^2 + GS)$$

из-за сохранения наблюдений и контактов.

В фактическом коде доступность каждого спутника/шлюза проверяется линейным просмотром соответствующих глобальных списков отказов. Если F_s^{all} — число записей satellite failure, а F_g^{all} — число gateway outage, добавляется худший член

$$\mathcal{O}(NF_s^{all} + GF_g^{all}).$$

Он может быть снижен индексированием отказов по ID и времени без изменения математики модели.

3 Статическая графовая модель

3.1 Физический граф и query-граф

На одном временном срезе строится граф

$$G_t = (V_t, E_t),$$

где вершины имеют тип *satellite*, *client* или *gateway*, а ребра — ISL или ground-satellite.

Для запроса конкретного клиента c физический неориентированный контакт преобразуется в ориентированный query-граф по ролям:

- c — единственный **SOURCE**;
- доступный спутник — **TRANSIT**;
- доступный шлюз — **TARGET**;
- остальные вершины — **BLOCKED**.

Дуга может выходить только из SOURCE/TRANSIT и входить только в TRANSIT/TARGET. Поэтому другой клиент не становится транзитным узлом, а шлюз не используется как relay.

Построение query-графа: $\mathcal{O}(V + E)$.

3.2 Покрытие и достижимость — разные величины

Геометрическое покрытие клиента:

$$\mathcal{V}_c = \{s : V_{c,s} = 1, A_s = 1\}.$$

Сервисная достижимость:

$$\mathcal{G}_c = \{g \in \text{Gate} : g \in \text{Reach}_{G_t}(c)\}.$$

Сервис доступен тогда и только тогда, когда

$$R_c = \mathbf{1}[\mathcal{G}_c \neq \emptyset].$$

Это принципиально отличается от $\mathcal{V}_c \neq \emptyset$: видимый спутник может оказаться тупиком без ISL-пути до шлюза.

Обычная достижимость после построения query-графа имеет $\mathcal{O}(V + E)$.

3.3 Допустимые первые спутники

Пусть $N^+(c)$ — исходящие соседи клиента. Используемым ingress считается только спутник $s \in N^+(c)$, из которого достижим хотя бы один TARGET:

$$\mathcal{I}_c = \{s \in N^+(c) \cap \text{Sat} : \text{Reach}(s) \cap \text{Gate} \neq \emptyset\}.$$

Текущая реализация запускает обход достижимости отдельно из каждого первого соседа. Поэтому

$$T_{\text{ingress}} = \mathcal{O}(d_c(V + E)).$$

Математически тот же результат можно получить дешевле обратным обходом от всех шлюзов, но текущий код использует именно повторные descendants-запросы.

3.4 Диагностика отсутствия маршрута

Если $R_c = 0$, причина выбирается *последовательным приоритетным классификатором*: берётся первая истинная строка из списка

1. no_visible_satellite, если $\mathcal{V}_c = \emptyset$;
2. gateway_unavailable, если нет ни одного доступного gateway;
3. no_gateway_contact, если доступные gateway есть, но ни один не имеет ground-satellite контакта;
4. isl_disconnected во всех остальных недостижимых случаях.

Это не четыре независимых булевых признака, а взаимоисключающий результат за счёт порядка проверки.

4 Маршрутизация

4.1 Детерминированный Дейкстра

Для неотрицательной аддитивной стоимости ребра $w(e)$ алгоритм минимизирует

$$C(P) = \sum_{e \in P} w(e).$$

Очередь имеет ключ

$$(C(P), |P| - 1, P),$$

где последний компонент — лексикографически сравниваемая последовательность ID узлов. Это делает результат воспроизводимым при равенстве стоимости: сначала меньше hop count, затем лексикографически меньший путь.

Номинальная сложность одного поиска:

$$D(V, E) = \mathcal{O}((V + E) \log V).$$

В Python-реализации полный tuple пути копируется и участвует в tie-break, поэтому в патологических случаях есть дополнительный фактор, зависящий от длины пути; приведённая оценка соответствует стандартной графовой части алгоритма.

4.2 Минимум переходов

Стоимость каждого ребра:

$$w(e) = 1.$$

Для пути P :

$$H(P) = |P| - 1.$$

Качество маршрута состоит из одной минимизируемой координаты $H(P)$.

Поскольку каждый достижимый шлюз обрабатывается отдельным запуском shortest path, при W_r достижимых target:

$$T_{minhop} = \mathcal{O}(W_r D(V, E)).$$

4.3 Минимальная геометрическая длина

Стоимость ребра:

$$w(e) = d_e,$$

а полная длина маршрута

$$L(P) = \sum_{e \in P} d_e.$$

Качество:

$$Q_{dist}(P) = (L(P), H(P)),$$

обе координаты минимизируются лексикографически; затем используются deterministic target/path tie-breaks.

Сложность также

$$T_{mindist} = \mathcal{O}(W_r D(V, E)).$$

4.4 Структурированное качество и частичный порядок

Для координаты q задаётся направление $\uparrow$ (maximize) или $\downarrow$ (minimize). Отношение “не лучше” определяется так:

$$x \preceq y \iff \begin{cases} x \leq y, & \uparrow, \\ x \geq y, & \downarrow. \end{cases}$$

Для вектора качества $Q = (q_1, \dots, q_m)$ используется произведённый Pareto-порядок:

$$Q \preceq Q' \iff \forall j : q_j \preceq q'_j.$$

Поэтому две альтернативы могут быть несравнимы. Для фактического выбора стратегии вводят отдельное лексикографическое полное продолжение порядка.

4.5 Отказоустойчивый маршрут resilient_distance

Эта стратегия реализована, но **не входит в reference-план по умолчанию**. Она активируется планом reference_case_with_resilient_routing.

Для каждого доступного спутника s сначала удаляется s и находится лучший по длине резервный путь до любого target:

$$B(s) = \begin{cases} \min_{P \subseteq G - \{s\}} L(P), & \text{если путь существует,} \\ P_\infty, & \text{иначе,} \end{cases}$$

где в коде

$$P_\infty = 1 + \sum_{e \in E} d_e.$$

Также

$$b(s) = \mathbf{1}[\text{после удаления } s \text{ сервис сохраняется}].$$

Для основного пути P :

$$\text{survive}(P) = \min_{s \in P \cap \text{Sat}} b(s), \quad \text{backup}(P) = \max_{s \in P \cap \text{Sat}} B(s).$$

Для пути без спутников используются значения 1 и 0 соответственно.

Полный критерий выбора:

$$\max \text{survive}(P), \quad \min \text{backup}(P), \quad \min L(P), \quad \min H(P),$$

затем deterministic tie-break по target/path.

Алгоритм решает первые две координаты без весовой суммы. Сначала, если существует путь только через спутники с $b(s) = 1$, все хрупкие спутники исключаются. Затем перебираются возрастающие пороги $\tau \in \{B(s)\}$; при каждом τ исключаются спутники с $B(s) > \tau$. Первый порог, при котором существует путь, даёт минимально возможный bottleneck $\max B(s)$.

Поскольку вычисление всех $B(s)$ требует до $F \cdot W_r$ запусков Дейкстры, а пороговый этап — ещё до $F \cdot W_r$ запусков,

$$T_{\text{resilient}} = \mathcal{O}(FW_r D(V, E)).$$

5 Мгновенная отказоустойчивость

5.1 Спутниковая vertex-connectivity

Требуется считать только спутники допустимыми точками отказа, не разрезая клиента и шлюзы. Для этого каждый узел v расщепляется:

$$v_{in} \rightarrow v_{out}.$$

Ёмкость внутреннего ребра:

$$c(v_{in}, v_{out}) = \begin{cases} 1, & v \in \text{Sat}, \\ \infty, & v \notin \text{Sat}. \end{cases}$$

В реализации конечная замена бесконечности равна $F + 1$, чего достаточно, чтобы любой спутниковый разрез был дешевле разреза endpoint.

Каждая допустимая query-дуга $u \rightarrow v$ превращается в

$$u_{out} \rightarrow v_{in}$$

с ёмкостью ∞ . Все шлюзы соединяются с общим sink также бесконечной ёмкостью.

Тогда

$$\kappa_c = \text{mincut}(c_{out}, \text{sink})$$

равна максимальному числу путей к сервису, внутренне непересекающихся по спутникам, а спутники, чьи внутренние рёбра попали в минимальный разрез, образуют один minimum satellite cut.

Для split-графа $V' = \Theta(V)$ и $E' = \Theta(V + E + W)$:

$$M(V, E) = \mathcal{O}(V'^2 \sqrt{E'})$$

для текущего default preflow-push NetworkX, плюс предварительные проверки достижимости.

5.2 Проверка N-1 перебором отказов

Если клиент изначально недостижим, свойство N-1 считается ложным. Иначе для каждого s из failure domain проверяется достижимость в $G - \{s\}$:

$$\mathcal{K}_c = \{s : \text{Reach}_{G-\{s\}}(c) \cap \text{Gate} = \emptyset\}.$$

Мгновенная N-1 устойчивость:

$$N1_c = \mathbf{1}[R_c = 1 \wedge \mathcal{K}_c = \emptyset].$$

Стоимость самого перебора критических спутников:

$$\mathcal{O}(F(V + E)).$$

Это отдельная операция от min-cut: код считает и κ_c , и явный список одиночных отказов.

6 Мгновенный контрфактический анализ отказа спутника

Для каждого доступного спутника s строится контрфактическая сеть

$$G^{(-s)} = G - \{s\}$$

и заново анализируется каждый клиент.

Для клиента c фиксируются, в частности:

$$\begin{aligned}\Delta_{c,s}^{suc} &= \mathbf{1}[R_c = 1 \wedge R_c^{(-s)} = 0], \\ \Delta_{c,s}^{vis} &= \mathbf{1}[|\mathcal{V}_c| > 0 \wedge |\mathcal{V}_c^{(-s)}| = 0], \\ \Delta I_{c,s} &= \max(0, |\mathcal{I}_c| - |\mathcal{I}_c^{(-s)}|), \\ \Delta G_{c,s} &= \max(0, |\mathcal{G}_c| - |\mathcal{G}_c^{(-s)}|), \\ \Delta \kappa_{c,s} &= \max(0, \kappa_c - \kappa_c^{(-s)}).\end{aligned}$$

Для каждой стратегии маршрутизации сохраняются путь до/после, факт потери маршрута, факт изменения node-path и отношение качества *after* к *before*.

6.1 Статический рейтинг критичности

Спутники сортируются лексикографически по убыванию вектора ущерба:

$$\left(|L_s|, |V_s|, \sum_c \Delta I_{c,s}, \sum_c \Delta G_{c,s}, \sum_c \Delta \kappa_{c,s}, \#route_lost, \#route_changed \right),$$

где L_s — клиенты, полностью потерявшие сервис, V_s — клиенты, потерявшие всю геометрическую видимость. Последний tie-break — ID спутника.

Это не взвешенная сумма и не centrality score.

6.2 Главная стоимость статического анализа

Обозначим через Q стоимость вычисления всех включённых маршрутов одного клиента, а через M — стоимость min-cut. Для reference-плана

$$Q = \mathcal{O}(KW_r D(V, E)), \quad K = 2.$$

Базовый анализ всех клиентов включает маршруты, connectivity и явные N-1 проверки:

$$\mathcal{O}(C(Q + M + F(V + E) + d_c(V + E))).$$

Затем failure-impact проходит по каждому s и каждому c . В контрфактическом анализе повторно считаются маршруты и connectivity, но не выполняется вложенный N-1 перебор:

$$T_{failure} = \mathcal{O}(FC(Q + M + d_c(V + E))).$$

Именно этот член обычно доминирует стоимость одного полного StaticModel.analyze().

7 Динамическая модель

7.1 Дискретная временная сетка

Сетка задаётся

$$\mathcal{T} = \{t_k = t_0 + k\Delta t \mid k = 0, \dots, T-1\},$$

причём

$$T = \frac{t_{end} - t_0}{\Delta t}, \quad [t_k, t_k + \Delta t)$$

является интервалом, представляемым отсчётом t_k . Правая граница всего горизонта не включается.

На каждом t_k выполняется полный конвейер

$$\text{SpatialSnapshot}(t_k) \rightarrow \text{StaticNetwork}(t_k) \rightarrow \text{StaticAnalysis}(t_k).$$

7.2 Availability и outage-интервалы

Для булевой временной серии $a_k \in \{0, 1\}$:

$$A = \frac{1}{T} \sum_{k=0}^{T-1} a_k.$$

Доступные и недоступные интервалы — максимальные непрерывные последовательности одинакового булева значения на дискретной сетке. Для набора outage-интервалов $\mathcal{O}$:

$$\begin{aligned} T_{out} &= \sum_{I \in \mathcal{O}} |I|, \\ T_{out}^{max} &= \max_{I \in \mathcal{O}} |I|, \\ T_{out}^{avg} &= \begin{cases} 0, & \mathcal{O} = \emptyset, \\ T_{out}/|\mathcal{O}|, & \text{иначе.} \end{cases} \end{aligned}$$

Для клиента отдельно вычисляются:

- visibility availability из $\mathbf{1}[|\mathcal{V}_c(t_k)| > 0]$;
- service availability из $R_c(t_k)$;
- выполнение целевого уровня $A_c \geq A_{target}$.

7.3 Статистика причин отсутствия маршрута

Для причины r :

$$\begin{aligned} n_r &= \#\{k : \text{reason}_c(t_k) = r\}, & T_r &= n_r \Delta t, \\ f_r^{period} &= \frac{n_r}{T}, & f_r^{outage} &= \frac{n_r}{T - N_{avail}} \end{aligned}$$

при ненулевом числе outage-отсчётов; иначе $f_r^{outage} = 0$.

7.4 Эпизоды маршрута и переключения

Логическая идентичность маршрута в reference-политике задаётся точным равенством упорядоченного списка узлов:

$$P(t_i) \equiv P(t_j) \iff \text{nodes}(P(t_i)) = \text{nodes}(P(t_j)).$$

Изменение непрерывных длин рёбер не создаёт switch, пока node-path тот же.

Route episode — максимальная последовательность соседних отсчётов с существующим и логически одинаковым маршрутом.

Direct switch в t_k фиксируется только если в t_{k-1} и t_k маршруты существуют и различны. Если между ними был outage, исчезновение и последующее восстановление маршрута не считается прямым A→B switch.

Для существующих маршрутов считаются min/max/mean hop count, длина и каждая координата route quality.

7.5 Временная N-1 устойчивость

Для клиента:

$$A_c^{N1} = \frac{1}{T} \sum_k N1_c(t_k).$$

Также агрегируется временная серия $\kappa_c(t_k)$ и для каждого спутника — сколько отсчётов он входил в явный список мгновенно критических спутников $\mathcal{K}_c(t_k)$.

8 Периодическая критичность спутников

Это наиболее содержательная динамическая аналитика. Важно, что здесь не запускается новый статический анализ: динамический слой собирает контрфактические результаты, которые уже были рассчитаны внутри каждого временного среза.

8.1 Контрфактическая доступность

Для спутника s строится серия

$$R_c^{(-s)}(t_k).$$

Если s в данном кадре не был участником failure domain (например, неактивен), контрфактическое состояние принимается равным baseline. Если был — используется уже вычисленный SatelliteFailureImpact.

$$A_c = \frac{1}{T} \sum_k R_c(t_k), \quad A_c^{(-s)} = \frac{1}{T} \sum_k R_c^{(-s)}(t_k),$$

$$\Delta A_{c,s} = A_c - A_c^{(-s)} \geq 0.$$

Код явно проверяет монотонность: удаление спутника не должно увеличивать service availability.

8.2 Outage-ущерб

Для каждого c, s :

$$\begin{aligned}\Delta T_{c,s}^{out} &= T_c^{out,(-s)} - T_c^{out}, \\ \Delta T_{c,s}^{max} &= \max(0, T_c^{max,(-s)} - T_c^{max}), \\ \Delta N_{c,s}^{out} &= N_c^{out,(-s)} - N_c^{out}.\end{aligned}$$

Последняя величина знаковая: один дополнительный отказ может слить несколько baseline-outage в один длинный интервал и тем самым уменьшить число outage-эпизодов.

Спутник “роняет клиента ниже SLA” тогда и только тогда, когда

$$A_c \geq A_{target} \quad \wedge \quad A_c^{(-s)} < A_{target}.$$

8.3 Интегральные потери резервирования

Код интегрирует дискретные потери, умножая сумму по отсчётам на Δt :

$$\begin{aligned}L_{c,s}^{vis} &= \Delta t \sum_k \mathbf{1}[\text{полная visibility потеря}], \\ L_{c,s}^{satvis} &= \Delta t \sum_k \max(0, |\mathcal{V}_c| - |\mathcal{V}_c^{(-s)}|), \\ L_{c,s}^{ing} &= \Delta t \sum_k \max(0, |\mathcal{I}_c| - |\mathcal{I}_c^{(-s)}|), \\ L_{c,s}^{gw} &= \Delta t \sum_k \max(0, |\mathcal{G}_c| - |\mathcal{G}_c^{(-s)}|), \\ L_{c,s}^{\kappa} &= \Delta t \sum_k \max(0, \kappa_c - \kappa_c^{(-s)}).\end{aligned}$$

Единицы последних четырёх величин соответственно имеют смысл satellite-seconds, satellite-seconds, gateway-seconds и path-seconds; это интеграл “сколько единиц избыточности потеряно и как надолго”.

8.4 Влияние на маршруты

Для каждой стратегии строится полная контрфактическая route timeline, после чего заново вычисляются:

- availability маршрута;
- число direct switches;
- число и длительность отсчётов, где маршрут потерян;
- число и длительность отсчётов, где node-path изменился;
- распределение отношений качества {better, equal, worse, incomparable} для пар, где маршруты существуют до и после.

Тем самым switch count контрфактического режима считается по целой временной последовательности, а не суммой локальных событий.

8.5 Периодический рейтинг спутников

Для каждого спутника строится summary, после чего сортировка лексикографически максимизирует ущерб в порядке:

(#clients falling below target, total additional outage, $\max_c \Delta A_{c,s}$, $\max_c \Delta T_{c,s}^{max}$, geometric visibility loss, ingress loss, gateway loss, connectivity loss, route lost samples, route changed samples, positive switch increase).

И снова это не свёртка в один непрозрачный score.

8.6 Сложность динамического анализа

Пусть T_{static} — стоимость полного статического анализа одного кадра, включая failure impacts. Тогда основное вычисление:

$$T_{dynamic} = \mathcal{O}(T(T_{3D} + T_{static})).$$

Последующая агрегация обычных client metrics линейна по трассе:

$$\mathcal{O}(TCK).$$

Сбор периодической критичности использует уже готовые статические impacts и в грубом виде имеет

$$\mathcal{O}(TFCK)$$

по объёму просматриваемых локальных контрфактических результатов. На практике дорогой частью остаётся их предварительное получение внутри T статических кадров.

Память велика, потому что DynamicAnalysis сохраняет *полный* spatial+static frame для каждого t_k . Нижняя оценка хранения самой трассы — $\Omega(T(V + E))$, а с сохранёнными статическими failure impacts существенно выше: порядка $\mathcal{O}(TFC)$ структур результатов плюс маршруты и наблюдения.

9 Сравнение вариантов

Варианты допускаются к сравнению только на одинаковой временной сетке. Один вариант объявляется baseline.

Для любой числовой метрики:

$$\Delta x = x_{variant} - x_{baseline}.$$

Знак не нормализуется под “хорошо/плохо”: например, $\Delta availability > 0$ — улучшение, а $\Delta outage > 0$ — ухудшение.

9.1 Итоговые метрики варианта

По клиентам вычисляются:

$$\begin{aligned}
 A_{min} &= \min_c A_c, \\
 A_{mean} &= \frac{1}{C} \sum_c A_c, \\
 V_{min} &= \min_c V_c, \\
 T_{sum}^{out} &= \sum_c T_c^{out}, \\
 T_{worst}^{out} &= \max_c T_c^{max}, \\
 N_{sum}^{switch} &= \sum_c N_{c,primary}^{switch}, \\
 N1_{min} &= \min_c A_c^{N1}.
 \end{aligned}$$

Дополнительно сохраняются доля времени, когда видимы все клиенты, доля времени, когда достижимы все клиенты, число клиентов выше target и top-ranked critical satellite.

9.2 Различие маршрутов между вариантами

Для общей стратегии на одинаковых временных отсчётах:

$$D_{path} = \sum_k \mathbf{1}[P_b(t_k) \neq P_v(t_k)], \quad f_{path} = \frac{D_{path}}{T}.$$

Если маршрут существует только в одном варианте, отсчёт считается различающимся.

Сложность сравнения одной пары вариантов в основном линейна по уже готовым результатам:

$$\mathcal{O}(CTK + F + P),$$

где P — число сравниваемых конфигурационных параметров. Для M вариантов относительно одного baseline — $\mathcal{O}((M-1)(CTK + F + P))$.

10 Что является заменяемым математическим компонентом

Модель специально разделяет данные результата и вычислительные политики. Без изменения остальных уровней можно заменить следующие математические определения:

Граница	Что можно заменить
Trajectory provider	Круговую орбиту на SGP4/TLE, численный интегратор и т.п.; наружу нужны состояние $s(t)$ и группа/плоскость.
Ground geometry	Сферическую модель координат наземной точки.

Граница	Что можно заменить
Ground visibility	Порог угла места на другую геометрическую видимость.
Ground link policy	Правило “виден $\Rightarrow$ линк” на link budget, терминальные ограничения и т.п.
ISL candidate source	Полный $\Theta(S^2)$ перебор на spatial index / broad-phase pruning.
ISL link policy	Порог дальности + Earth occlusion на иной физический критерий.
Coverage policy	Определение локального покрытия клиента.
Reachability policy	Типы источников/target/transit и допустимые рёбра.
No-route classifier	Приоритет и набор диагностических причин.
Route cost/strategy	Hop count, distance, многокритериальную или другую стратегию выбора.
Graph algorithms	Реализацию reachability, shortest path, connectivity за интерфейсом.
Failure domain	Набор отказов: только активные спутники сейчас; можно определить другой домен.
Criticality ranking	Порядок ранжирования raw impact vectors.
Route identity	Что считать тем же логическим маршрутом во времени.
Route comparison	Что считать тем же маршрутом между вариантами.

Важно: replacement boundary не означает, что произвольная замена автоматически физически корректна. Она означает, что архитектура отделила конкретное математическое определение от остальных результатов и позволяет заменить его без переписывания соседних компонентов.

11 Что текущий backend математически не моделирует

В коде нет переменных трафика и пропускной способности, поэтому сейчас **невозможно** из существующей модели корректно получить “нагрузку на спутник после отказа”, “пиковую утилизацию”, congestion или очередь. Маршрут — это факт существования/выбора пути, а не поток.

Не моделируются:

- спрос $d_c(t)$ клиента или матрица traffic demands;
- capacity $C_e(t)$ линков и capacity узлов;
- распределение потоков по маршрутам и shared-link contention;
- очереди, packet loss, scheduling, retransmission;
- throughput и load/utilization;
- store-carry-forward / DTN;
- энергия и ограничения терминалов;
- задержка как физическая величина с processing/queueing; геометрическая длина используется только как монотонный proxy propagation delay.

Чтобы считать пиковую нагрузку после отказа, минимально нужен потоковый слой. Например, при фиксированном demand $d_c(t)$ и единственном выбранном пути $P_c(t)$ нагрузка ребра могла бы быть определена как

$$\ell_e(t) = \sum_c d_c(t) \mathbf{1}[e \in P_c(t)], \quad u_e(t) = \frac{\ell_e(t)}{C_e(t)},$$

а после отказа s — повторно по $P_c^{(-s)}(t)$. Тогда естественная метрика пикового перераспределения:

$$\Delta \nu_s^{peak} = \max_{t,e} \left(\nu_e^{(-s)}(t) - \nu_e(t) \right).$$

Но без d_c и C_e такая величина в текущем backend не определена и подмена её числом маршрутов была бы другой моделью.

12 Сводная таблица сложности

Операция	Оценка текущей реализации
Позиции спутников	$\mathcal{O}(N)$ плюс линейные проверки outage-списков.
ISL broad phase	$\Theta(S^2)$.
Ground observations	$\Theta(GS)$.
Один SpatialSnapshot	$\Theta(N + S^2 + GS)$ без стоимости линейных outage lookup.
Построение query-графа	$\mathcal{O}(V + E)$.
Reachable targets	$\mathcal{O}(V + E)$.
Viable first hops	$\mathcal{O}(d_c(V + E))$.
Один shortest path	номинально $\mathcal{O}((V + E) \log V)$.
Стандартная стратегия до любого gateway	$\mathcal{O}(W_r D(V, E))$.
Resilient-distance strategy	$\mathcal{O}(FW_r D(V, E))$.
Satellite connectivity	split graph + min-cut: $\mathcal{O}(V'^2 \sqrt{E'})$ для default preflow-push, плюс reachability checks.
Явная N-1 проверка клиента	$\mathcal{O}(F(V + E))$.
Все static failure impacts	$\mathcal{O}(FC(Q + M + d_c(V + E)))$; обычно доминируют.
Полная динамика	$\mathcal{O}(T(T_{3D} + T_{static}))$.
Temporal aggregation	$\mathcal{O}(TCK)$; period criticality aggregation $\mathcal{O}(TFCK)$.
Сравнение пары вариантов	$\mathcal{O}(CTK + F + P)$ по уже рассчитанным анализам.

13 Краткие выводы по математике реализации

1. Геометрия прямых контактов реализована последовательно: круговая орбита, вращение Земли, elevation, строгие range/occlusion условия ISL.
2. Покрывтие и end-to-end сервис не смешиваются: видимость — локальное геометрическое свойство, достижимость — графовое.
3. Маршрутизация детерминирована; reference-plan реально считает два маршрута: minimum hops и minimum distance.

4. Отказоустойчивость анализируется двумя способами: min-cut даёт спутниковую связность, а явный перебор одиночных отказов — N-1 и критические спутники.
5. Самая дорогая часть — контрфактический статический анализ каждого доступного спутника для каждого клиента на каждом временном отсчёте.
6. Динамическая критичность математически содержательнее простого “сколько раз спутник был критическим”: она реконструирует периодическую availability, outage и route timeline без спутника.
7. Рейтинги в статике и динамике лексикографические и объяснимые; скрытого weighted score нет.
8. Текущая модель не является traffic/load model. Для пиковых нагрузок после отказов нужен отдельный потоковый слой с demand и capacity.

А Трассировка формул к backend-модулям

Математическая часть	Реализация
Круговая орбита, ECI→Earth-fixed	spatial3d/trajectory.py
Наземные координаты	spatial3d/kinematics.py
Elevation и closest segment approach	spatial3d/observations.py
Visibility, ground link, ISL condition, pair generation	spatial3d/contacts.py
Deployment/outage availability	spatial3d/availability.py
Сборка пространственного снимка	spatial3d/model.py
Роли query-графа и no-route classifier	static_model/policies.py
Reachability, viable ingress, Dijkstra, min-cut	static_model/networkx_engine.py
Route quality и route/failure result types	static_model/analysis.py
Minimum-hops, minimum-distance, resilient-distance	static_model/routing.py
Static N-1 и failure impacts	static_model/model.py
Reference route plan	static_model/plan.py
Time grid, availability, episodes, switches, period criticality	dynamic_model/model.py
Dynamic criticality ranking и route identity	dynamic_model/policies.py
Variant deltas и path comparison	variant_comparison/model.py
Case constants и mapping input→model	cosmo_a_json/adapter.py